Automating the CI/CD Pipeline for TechCorp's LLM Infrastructure

Module 1 Hands-On Project Report — Peer Review submission

Author: Luxna Ramnauth | Date: 3 October 2026

Before you submit: this report is built from the starter and solution files in the Module1_Deployment package. Anything marked **[FILL IN]** must come from your own runs (image sizes, pipeline timings, scan results, screenshots). They are left blank on purpose so no measurement in the report is invented. Figures labelled "reference" are quoted from the course files, not measured by you.

Executive summary

TechCorp's manual LLM deployments take 6 hours and caused three production incidents (about $200K lost revenue). This project replaces that process with an automated pipeline: an optimized multi-stage Docker image, a GitHub Actions workflow with Trivy security scanning, a blue-green deployment with health checks and automatic rollback, and Terraform-defined AWS infrastructure. The target is under 30 minutes end to end with zero downtime.

Metric	Manual (baseline)	Automated (this project)
Deployment time	6 hours	**[FILL IN]** min (target < 30 min)
Image size	~45 GB (template reference)	**[FILL IN]** GB (target < 20 GB; reference solution ~15 GB)
Production incidents	3 (≈ $200K lost revenue)	**[FILL IN]**
Downtime during update	Yes	**[FILL IN]** (target: none)

Requirement 1 — Optimized Docker container

Problems in the original template: single-stage build that ships build tools (cmake, git, vim, htop) in the final image; `COPY . .` before installing dependencies, so any code change re-runs pip; the full `cuda:12.1-devel` base image; no .dockerignore; and the app runs as root.

Optimized multi-stage Dockerfile

```
# ============================
# STAGE 1: Builder
# ============================
FROM nvidia/cuda:12.1-devel-ubuntu22.04 AS builder

# Set environment variables
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

# Install Python and build dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3.10 \
    python3-pip \
    python3-dev \
    build-essential \
    && rm -rf /var/lib/apt/lists/* \
    && apt-get clean

WORKDIR /build

# OPTIMIZATION: Copy requirements first for layer caching
# This layer only rebuilds when requirements.txt changes
COPY M1_requirements.txt ./requirements.txt
RUN pip3 install --no-cache-dir --target=/build/deps -r requirements.txt

# ============================
# STAGE 2: Runtime
# ============================
FROM nvidia/cuda:12.1-runtime-ubuntu22.04 AS runtime

# Set environment variables
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONPATH=/app/deps \
    MODEL_NAME=llama-3.1-8b \
    MODEL_VERSION=v1.0.0

# Install only runtime Python (no dev tools)
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3.10 \
    python3-distutils \
    && rm -rf /var/lib/apt/lists/* \
    && apt-get clean \
    && ln -s /usr/bin/python3.10 /usr/bin/python

# SECURITY: Create non-root user
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

WORKDIR /app

# Copy installed dependencies from builder
COPY --from=builder /build/deps /app/deps

# OPTIMIZATION: Copy application code last (changes most frequently)
COPY M1_app.py ./app.py

# Set ownership to non-root user
RUN chown -R appuser:appgroup /app

# SECURITY: Switch to non-root user
USER appuser

# Expose port
EXPOSE 8000

# Health check for container orchestration
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
```

```
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')" || exit 1

# Run the application
CMD ["python", "-m", "uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Layer caching strategy

- **Least to most frequently changing:** base image → system packages → `M1_requirements.txt` → pip install → application code last.
- A code-only change reuses the cached dependency layers; only the final COPY and later layers rebuild.
- **Builder vs. runtime:** compilers and -dev packages stay in the builder; the runtime stage gets only the installed packages (`--target=/build/deps`), the CUDA runtime and Python.
- **Security:** non-root `appuser`, `--no-install-recommends`, and a .dockerignore excluding .git, __pycache__, *.pyc, .env, tests/, docs/.

Size comparison

Image	Command	Size
Original template	docker build -f M1_Dockerfile.template -t techcorp-llm:orig .	**[FILL IN]**
Optimized	docker build -f Dockerfile -t techcorp-llm:v1 .	**[FILL IN]**
Reduction	(orig – optimized) / orig	**[FILL IN]** %

Health-check verification

```
docker run -p 8000:8000 techcorp-llm:v1
curl http://localhost:8000/health
docker inspect --format='{{.State.Health.Status}}' <container>
```

Result: **[FILL IN]** (paste output or screenshot showing status healthy).

Requirement 2 — CI/CD pipeline with security scanning

The workflow (`.github/workflows/deploy-llm.yml`) triggers on pushes to main (and on changes to app, Dockerfile and workflow paths), on pull requests, and manually with an environment choice (dev/staging/prod). It has four jobs: **build** → **security-scan** → **deploy** → **rollback** (on failure).

Job	Purpose	Key details
build	Build and push image	Buildx, ECR login, tags by commit SHA, layer cache via `type=gha`
security-scan	Trivy vulnerability scan	severity CRITICAL,HIGH; exit-code 1; ignore-unfixed; SARIF uploaded to GitHub Security tab
deploy	Blue-green deploy	ECS + CodeDeploy, health check loop, Slack notification
rollback	Automatic rollback	Runs if deploy fails; restores previous task definition

Build with layer caching and Trivy scan (excerpt)

```
      - name: Build and push Docker image
        id: build-image
        uses: docker/build-push-action@v5
        with:
          context: .
          file: ./Dockerfile
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
          build-args: |
            MODEL_VERSION=${{ github.sha }}
            BUILD_DATE=${{ github.event.head_commit.timestamp }}

...
      - name: Run Trivy vulnerability scanner
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: ${{ needs.build.outputs.image_uri }}
          format: 'sarif'
          output: 'trivy-results.sarif'
          severity: 'CRITICAL,HIGH'
          exit-code: '1'
          ignore-unfixed: true
          vuln-type: 'os,library'

      - name: Upload Trivy scan results to GitHub Security tab
        uses: github/codeql-action/upload-sarif@v2
        if: always()
        with:
          sarif_file: 'trivy-results.sarif'
```

Security scan configuration and results

Setting	Value
Severity gate	CRITICAL, HIGH (pipeline fails on any finding)
Unfixed CVEs	Ignored to reduce noise
Report	SARIF to GitHub Security tab + table summary in the log

Scan result: **[FILL IN]** (vulnerabilities found: critical **[FILL IN]** / high **[FILL IN]**; paste summary or screenshot).

Evidence of successful run: **[FILL IN]** (screenshot of green run in the Actions tab, or run URL).

Pipeline execution time

Stage	Duration
Build (cold cache)	**[FILL IN]**
Build (warm cache)	**[FILL IN]**
Security scan	**[FILL IN]**
Deploy + health check	**[FILL IN]**
Total	**[FILL IN]**

Credentials: the workflow reads AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, ALB_DNS and SLACK_WEBHOOK_URL from GitHub secrets. Never commit these values. The Trivy action is referenced as `@master` in the course solution; pin it to a released version or commit SHA for production.

Requirement 3 — Blue-green deployment configuration

Two identical target groups, **blue** (live) and **green** (new version), sit behind one Application Load Balancer. The new image is deployed to the idle group, validated, and only then does traffic move. If validation fails, traffic stays on (or returns to) the previous version.

Terraform: blue and green target groups with health checks

```
resource "aws_lb_target_group" "blue" {
  name        = "${var.project_name}-blue-${var.environment}"
  port        = 8000
  protocol    = "HTTP"
  vpc_id      = aws_vpc.main.id
  target_type = "instance"

  health_check {
    enabled             = true
    healthy_threshold   = 2
    unhealthy_threshold = 3
    timeout             = 5
    interval            = 30
    path                = "/health"
    port                = "traffic-port"
    matcher             = "200"
  }

  tags = {
    Name        = "${var.project_name}-blue-tg-${var.environment}"
    Deployment  = "blue"
  }
}
```

Health-check validation (deploy job)

```
- name: Validate deployment health
  id: health-check
  run: |
    ALB_DNS="${{ secrets.ALB_DNS }}"
    MAX_ATTEMPTS=30
    ATTEMPT=0

    echo "Waiting for deployment to be healthy..."

    while [ $ATTEMPT -lt $MAX_ATTEMPTS ]; do
      HEALTH=$(curl -s -o /dev/null -w "%{http_code}" "https://${ALB_DNS}/health" || echo "000")

      if [ "$HEALTH" = "200" ]; then
        echo " Health check passed!"

        # Verify response content
        RESPONSE=$(curl -s "https://${ALB_DNS}/health")
        STATUS=$(echo $RESPONSE | jq -r '.status')

        if [ "$STATUS" = "healthy" ]; then
          echo " Service is healthy and ready!"
          exit 0
        fi
      fi

      ATTEMPT=$((ATTEMPT + 1))
      echo "Attempt $ATTEMPT/$MAX_ATTEMPTS - Status: $HEALTH"
      sleep 10
    done

    echo " Health check failed after $MAX_ATTEMPTS attempts"
    exit 1
```

Automatic rollback trigger

- The `rollback` job has `if: failure()` and `needs: deploy`, so any failed deploy or health check triggers it.
- The deploy job saves the current task definition before changing anything; rollback runs `aws ecs update-service --task-definition <previous> --force-new-deployment`, waits for `services-stable`, then notifies Slack.
- Health gate: up to 30 attempts × 10 s (about 5 minutes); passes only on HTTP 200 and `status == healthy`.

Zero-downtime evidence

Test method: **[FILL IN]** (e.g., run a request loop against /health during a deployment: `while true; do curl -s -o /dev/null -w "%{http_code}\n" https://$ALB/health; sleep 1; done`).

Result: **[FILL IN]** requests sent, **[FILL IN]** failed; non-200 responses during switchover: **[FILL IN]**.

Requirement 4 — Infrastructure as code

Terraform (>= 1.0, AWS provider ~> 5.0) defines the full environment so it can be recreated identically for dev, staging and prod.

File	Contents
M1_main.tf	Provider + default tags; VPC; public and private subnets; internet gateway; security groups (LLM instances and ALB); ECR repository with scan-on-push and lifecycle policy; ALB; blue and green target groups; IAM role, policy and instance profile. GPU EC2 instance block is provided commented out.
M1_variables.tf	project_name, environment (validated: dev/staging/prod), aws_region, availability_zones, vpc_cidr, allowed_ssh_cidrs, instance_type (g5.xlarge), gpu_ami_id, min/max instances, model_bucket, model_name, model_version, additional_tags.
M1_outputs.tf	VPC and subnet IDs, ECR URL/ARN, ALB DNS/ARN, blue/green target group ARNs, security group ID, IAM role/profile, deployment_info summary.

Environment parameterization

```
variable "environment" {
  description = "Environment name (dev, staging, prod)"
  type        = string
  default     = "dev"
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

# Per-environment usage
terraform plan  -var="environment=dev"
terraform plan  -var="environment=staging" -var="instance_type=g5.2xlarge"
terraform plan  -var="environment=prod"    -var="min_instances=2"
```

Resource tagging strategy

```
provider "aws" {
  region = var.aws_region
  default_tags {
    tags = {
      Project     = "TechCorp-LLM"
      Environment = var.environment
      ManagedBy   = "Terraform"
    }
  }
}
```

- Every resource inherits Project, Environment and ManagedBy tags, enabling cost reports by environment.
- Individual resources add Name (and Deployment = blue/green on target groups).
- Environment-aware safeguards: ALB deletion protection is enabled only in prod.

terraform plan output

Command: `terraform init && terraform validate && terraform plan`. Summary line ("Plan: N to add, 0 to change, 0 to destroy"): **[FILL IN]**. Paste the resource list or a screenshot.

Provisioning time: automated [FILL IN] vs. manual setup [FILL IN].

Review notes from the starter files: the starter SSH rule defaults to the VPC CIDR (good); the ALB security group exposes 80/443 to the internet, which is expected for a public LLM API. The Terraform S3 backend is commented out — enable it (with state locking) for team use. The ALB outputs use https:// but no HTTPS listener or certificate is defined in the starter, so add one before relying on those URLs.

Requirement 5 — Performance comparison report

Measure	Manual	Automated	Change
Total deployment time	6 h (360 min)	[FILL IN]	[FILL IN]
Build	—	[FILL IN]	—
Security scan	Not routinely done / manual	[FILL IN]	—
Deploy + health check	—	[FILL IN]	—
Infrastructure provisioning	[FILL IN]	[FILL IN]	[FILL IN]
Production incidents	3	[FILL IN]	[FILL IN]
Downtime per update	Yes	[FILL IN]	[FILL IN]
Image size	~45 GB (template ref.)	[FILL IN]	[FILL IN]
Resource utilization (CPU / RAM / GPU at idle and under load)	[FILL IN]	[FILL IN]	—

Error-rate comparison

Manual: three production incidents in the baseline period, about $200K lost revenue. Automated: the pipeline removes the manual steps most prone to error and adds gates that stop a bad release before users see it:

- Trivy blocks releases with HIGH/CRITICAL findings.
- Health checks must pass before traffic moves.
- Failure triggers automatic rollback to the previous task definition.
- Terraform and the SHA-tagged image make every deployment reproducible.

Business analysis

(Write 3–5 sentences after you have your numbers. Suggested structure:) The automated pipeline reduced deployment time from 6 hours to [FILL IN] minutes, saving about [FILL IN] hours per release. It removes the manual configuration errors behind the three prior incidents by adding scanning, health-check gating and automatic rollback. Updates now ship without downtime through blue-green switching. Remaining gaps before production: [FILL IN] (for example HTTPS listener and certificates, pinned action versions, remote Terraform state with locking, load testing under production traffic). Overall readiness for production use: [FILL IN].

Appendix — submission checklist

- Requirement 1: Dockerfile, size comparison, caching notes, health-check proof.
- Requirement 2: workflow YAML, Trivy config and results, successful-run evidence, timings.
- Requirement 3: blue-green config, health check, rollback trigger, zero-downtime evidence.
- Requirement 4: .tf files, terraform plan output, environment variables, tagging.
- Requirement 5: comparison table and 3–5 sentence analysis.
- Save as PDF; upload to Peer Review with a brief description; review peers' work.